PROJECT REPORT: Agent MedIC

Team: Enthusiast (Rudra + Het Patel) **College:** Dr. Kiran and Pallavi Patel Global University **Hackathon:** Agents of SigNoz by WeMakeDevs **Track:** 01 — AI & Agent Observability **Prize Target:** Apple MacBook (per team member) **Dates:** July 20 – July 26, 2026

---

1. Problem Statement

AI agents today are black boxes. When latency spikes, costs explode, or an agent hallucinates in production, developers have no visibility. Traditional observability tools don't understand AI workflows.

Solution: Agent MedIC — a self-healing AI SRE agent that watches your infrastructure via SigNoz, auto-debugs failures, and heals them automatically, logging everything back into SigNoz.

---

2. What We Are Building

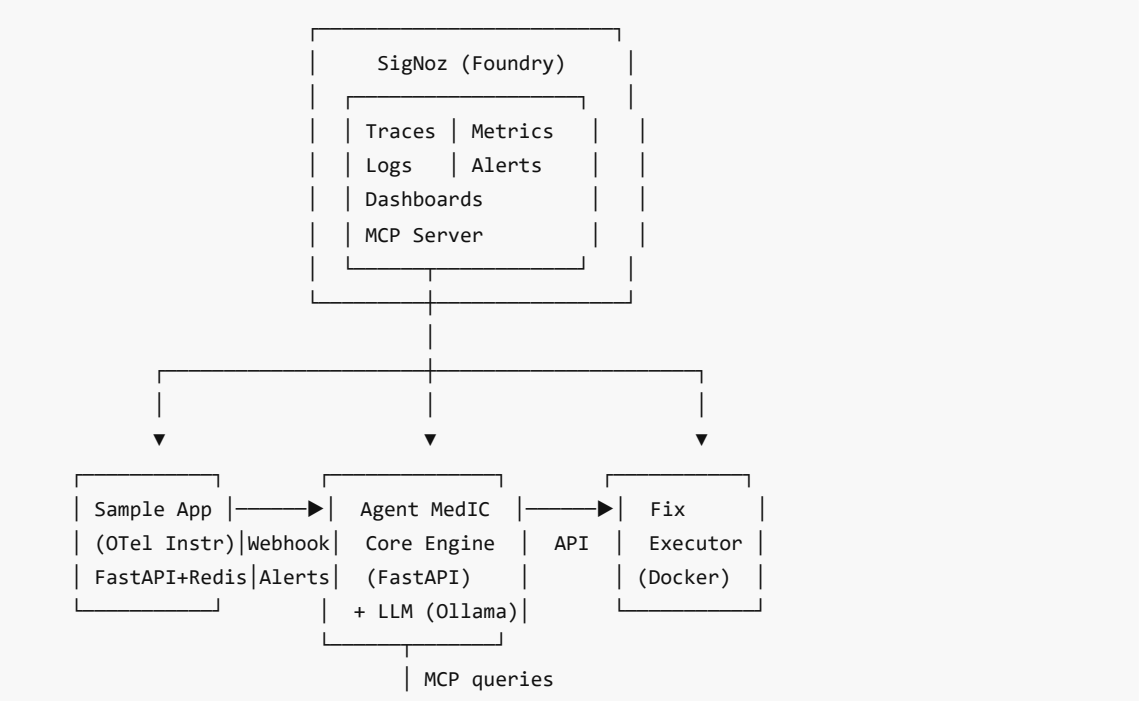
Agent MedIC ek AI agent hai jo continuously server/infrastructure ko monitor karta hai SigNoz ke through. Jab koi problem hoti hai (server crash, CPU high, database down), yeh agent:

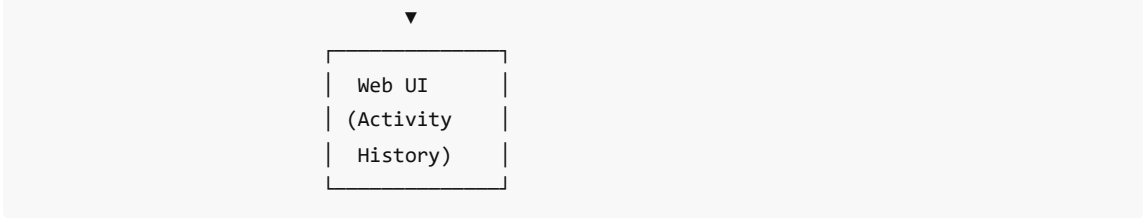
1. **Alert receive karta hai** SigNoz se (webhook)
2. **Investigation karta hai** — traces, metrics, logs fetch karta hai via SigNoz MCP
3. **Root cause find karta hai** using LLM (local Ollama model)
4. **Khud fix karta hai** — container restart, service scale, etc.
5. **Poora incident wapas SigNoz me log karta hai**

Real-life example: Website slow ho gayi. Agent MedIC detect karega ki Redis connection pool exhausted hai, Redis container restart karega, aur notification dega — "Issue found and fixed in 30 seconds."

---

3. Architecture





---

4. Tech Stack

Layer	Technology	Price
Observability Platform	SigNoz (self-hosted via Foundry)	Free (Open Source)
Instrumentation	OpenTelemetry Python SDK	Free
Agent Backend	Python FastAPI	Free
AI / LLM	Ollama (llama3.2 local)	Free — no API key needed
Agent Framework	LangChain / LangGraph	Free (Open Source)
MCP Integration	SigNoz MCP Server	Free
Auto-Fix	Docker SDK	Free
Frontend	Streamlit / HTML+JS	Free
Demo App	FastAPI + Redis + PostgreSQL (Docker)	Free
Deployment	Docker Compose	Free
Cloud	AWS ($100 free credits to all participants)	Free
CI/CD	GitHub Actions	Free

Key decision: Ollama lok karenge taaki koi LLM API cost na ho. Sab tools free hain.

---

5. Project Structure

```
agent-medic/
│
├── sample-app/                    # Buggy microservice
│   ├── app.py                     # FastAPI app
│   ├── instrument.py              # OpenTelemetry setup
│   ├── docker-compose.yml         # App + Redis + Postgres
│   └── requirements.txt
│
├── agent-medic/                   # AI agent
│   ├── main.py                    # FastAPI server
│   ├── alert_listener.py          # SigNoz webhook receiver
│   ├── mcp_client.py              # SigNoz MCP queries
│   ├── diagnosis_engine.py        # LLM analysis
│   ├── fix_executor.py            # Docker auto-fix
```

```
│   ├── incident_logger.py          # Log back to SigNoz
│   └── config.py                   # Config
│
├── web-ui/                         # Agent dashboard
│   └── index.html
│
├── casting.yaml                    # SigNoz Foundry config
├── casting.yaml.lock
├── docker-compose.yml              # Full stack deploy
├── README.md
└── demo-script.md
```

---

6. How It Works (Step-by-Step Flow)

```
Step 1: Sample App me bug trigger hota hai
        (500 error, high CPU, Redis down)

Step 2: SigNoz alert fire karta hai via webhook
        → Agent MedIC ko notification jaata hai

Step 3: Agent MCP se data query karta hai:
        "Show traces for last 5 min"
        "What is the error rate trend?"

Step 4: LLM (Ollama) analysis karta hai:
        "Redis connection pool exhausted"

Step 5: Agent fix execute karta hai:
        Docker SDK se Redis container restart

Step 6: Agent verify karta hai:
        Wapas SigNoz query → error rate zero? ✓

Step 7: Incident log karta hai SigNoz me:
        "Issue: Redis OOM → Action: Restart → Status: Resolved in 23s"

Step 8: Web UI update — "Incident #42 — Resolved"
```

---

7. Failure Scenarios for Demo

Scenario	Trigger	Agent Action
Redis Cache Crash	Stop Redis container	Detect → Restart Redis → Verify
CPU Spike / Memory Leak	Load test (1000 req/sec)	Detect CPU>80% → Scale replicas → Verify
Database Timeout	Stop PostgreSQL	Detect timeout in traces → Restart DB → Verify

Random 500 Errors	Code throws error	Correlate error rate → Isolate endpoint → Auto-rollback

---

8. 7-Day Execution Plan

Day	Date	Tasks	Owner
Day 1	Jul 20	SigNoz Foundry install + OTel instrumentation	Rudra + Het
Day 2	Jul 21	Alert pipeline (webhook → agent)	Rudra
Day 3	Jul 22	MCP client + LLM diagnosis engine	Het
Day 4	Jul 23	Auto-fix executor + full integration	Rudra + Het
Day 5	Jul 24	Web UI + SigNoz dashboards + polish	Rudra + Het
Day 6	Jul 25	Demo video + README + Blog post	Rudra + Het
Day 7	Jul 26	Final submission + Social media	Rudra + Het

---

9. Judging Criteria & Strategy

Criteria	Weight	Our Strategy
Potential Impact	20%	Real SRE problem — har company ko chahiye
Creativity & Innovation	20%	Self-healing via MCP+LLM — novel approach
Technical Excellence	20%	LangGraph, OTel, MCP, Docker SDK — solid stack
Best Use of SigNoz	20%	Traces + Metrics + Logs + Alerts + MCP + Dashboards
User Experience	10%	Clean web UI, one-click Foundry deploy
Presentation Quality	10%	Polished demo video + README + blog

---

10. Side Tracks (Extra Prizes)

Side Track	Prize	Action Plan
Best Blog	LEGO Ferrari SF-24 ($250)	"How We Built Self-Healing AI SRE Agent with SigNoz" on Dev.to
Social Buzz	Exclusive Swag	Daily progress posts on X/LinkedIn @wemakedevs @SigNozHQ
AWS Credits	$5K/$3K/$2K	Host everything on AWS with free $100 credits

---

11. Why We Will Win

1. **Maximum SigNoz usage** — traces, metrics, logs, alerts, MCP, dashboards — sab kuch
2. **Real-world problem** — AI agent observability + SRE automation
3. **Free tech stack** — Ollama + OpenSource tools, zero cost

4. **Beautiful demo** — Alert → Diagnose → Fix → Log, full cycle dikhega
5. **Reproducible** — `casting.yaml` + Foundry = judges 5 min me run kar sakte hain

---

Prepared for Agents of SigNoz Hackathon by Team Enthusiast Rudra & Het Patel — Dr. Kiran and Pallavi Patel Global University